

Ponteiros, Referências e Estruturas Dinâmicas

A estrutura da memória em Python

Professora Kadidja Valéria



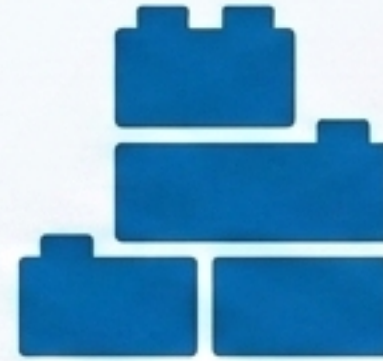
Conteúdo da aula



A Ilusão da
Memória
(Variáveis e
Referências)



A Natureza dos
Objetos
(Mutabilidade)



Construção
Dinâmica
(O conceito de
Node)



Mão na Massa
(O Sistema de
Clínica)

Desconstruindo o modelo mental das variáveis



Em Python, a variável não é uma caixa que guarda um valor.



A variável é um **nome** associado (referência) a um **objeto independente** na memória.

Como as linguagens enxergam a memória?

O Mundo do C/C++ (Ponteiros)



```
int *p = &idade;
```

Trabalha diretamente com o endereço físico da memória. Controle total, mas com alto risco de falhas críticas.

O Mundo do Python (Referências)



Ponteiros explícitos não existem. Trabalhamos com referências a objetos. O interpretador gerencia o endereço físico nos bastidores.

A identidade dos objetos

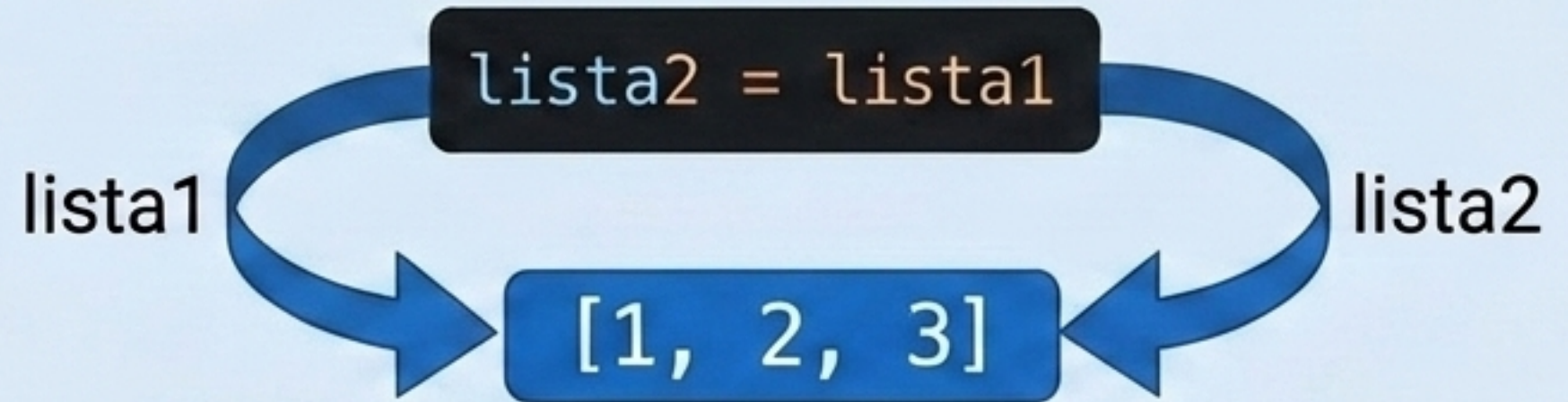
```
a = 5  
b = a  
print(id(a)) # Saída: 140728...  
print(id(b)) # Saída: 140728...
```



A função `id()` revela o DNA (endereço virtual) do objeto. Variáveis diferentes frequentemente apontam para o mesmo lugar exato!

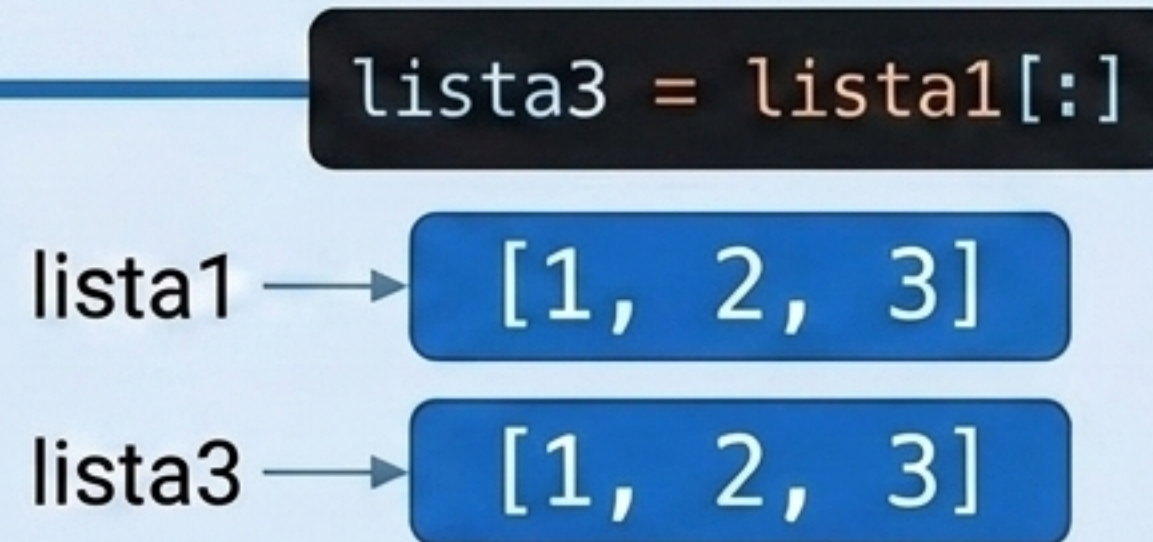
Referência × Cópia

Cenário A (Compartilhamento)



Alterar uma, altera a outra. O `id()` é idêntico.

Cenário B (Cópia Independente)



Um novo objeto nasce na memória com outro `id()`. Mudanças são isoladas.

O teste de resiliência: Mutabilidade

Objetos Mutáveis (Listas, Dicionários)

O objeto aceita a mudança interna.



Modificar a estrutura **altera o objeto original**.
O **id()** continua exatamente o mesmo.

Objetos Imutáveis (Inteiros, Strings)

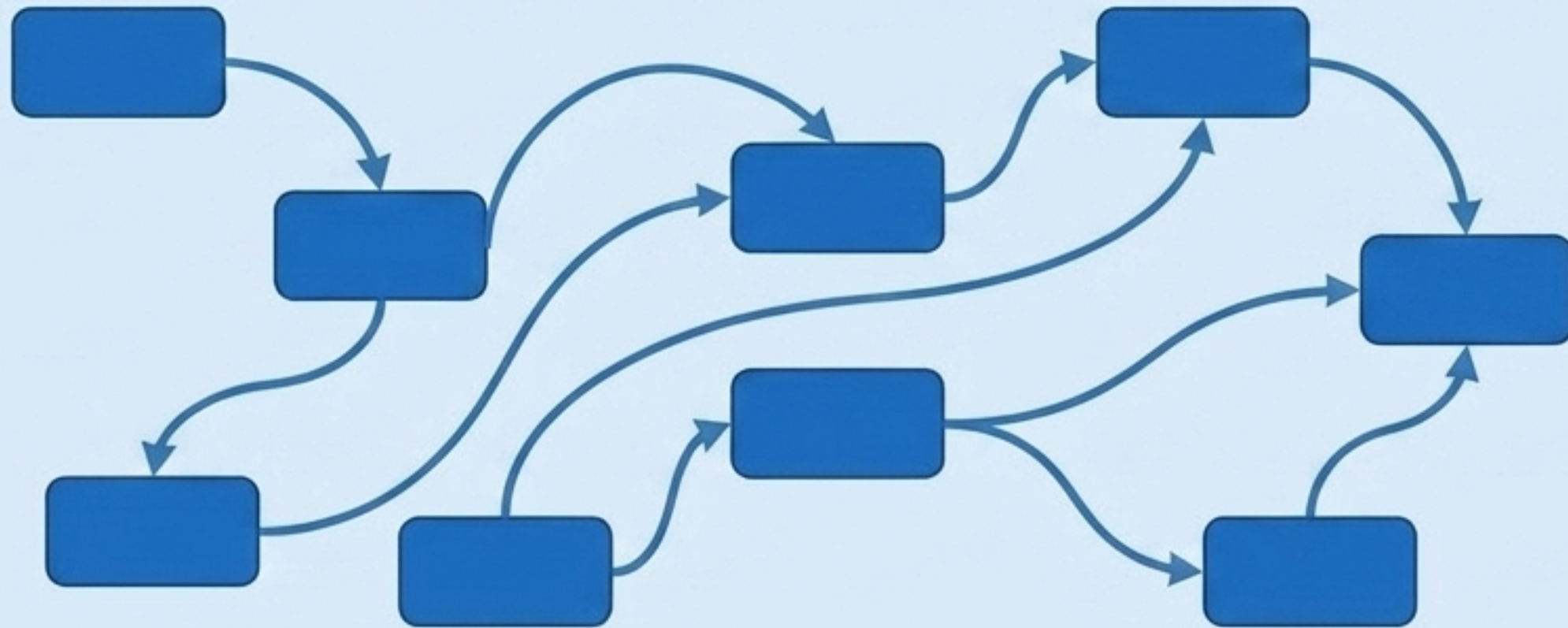
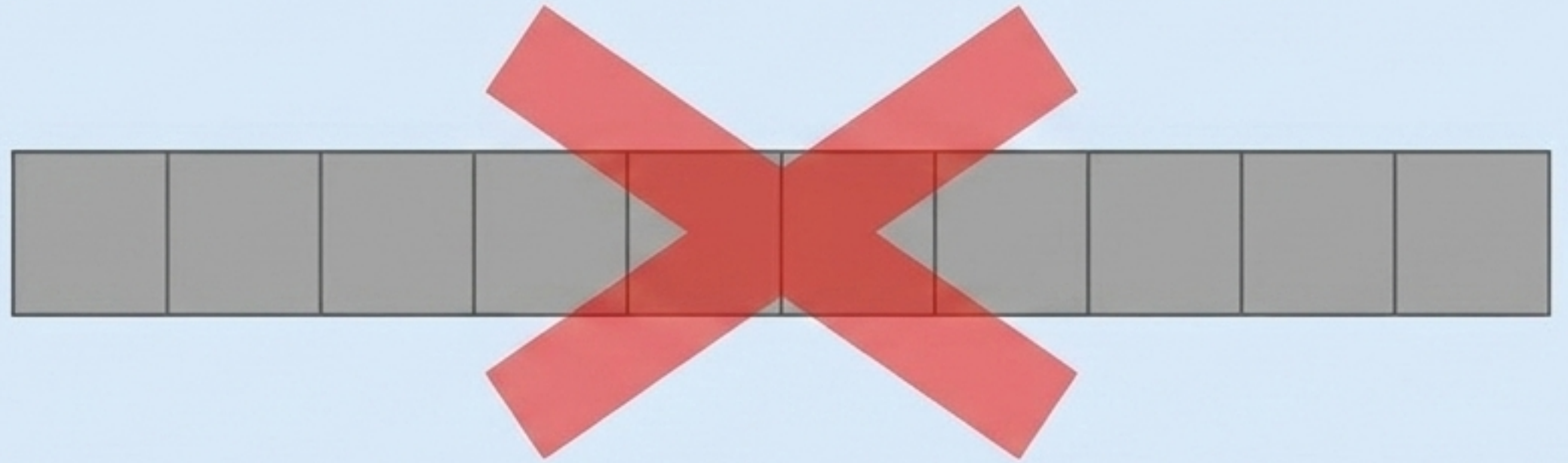
O objeto é rebelde e não muda.



O Python cria um **NOVO** objeto e **move a etiqueta** para ele. O **id()** é **alterado**!

O surgimento das estruturas dinâmicas

E se nossos dados **não precisarem morar em blocos contíguos** (vizinhos) na memória?



Estruturas Dinâmicas crescem e diminuem sob demanda. O segredo? Um objeto só precisa guardar a referência de onde está o próximo.

A anatomia de um Nó (Node)

```
class Node:  
    def __init__(self, dado):  
        self.dado = dado  
        self.proximo = None
```



O atributo proximo é a nossa âncora, a referência que conectará a corrente.

Forjando a corrente: A Lista Encadeada

```
no1 = Node(10)  
no2 = Node(20)  
no1.proximo = no2
```



É assim que a magia acontece. Não movemos os objetos; apenas apontamos as etiquetas na direção certa.

O Desafio do Mundo Real: Sistema da Clínica

O Paciente: Nome, idade, nível de prioridade.

A Dinâmica: Pacientes chegam de forma contínua e saem quando são atendidos.

O Contrato (Operações):

1. Adicionar paciente
2. Atender (remover o primeiro)
3. Listar espera
4. Checar se está vazia

A Arquitetura da Solução

Classe Fila:

// O Gerenciador: não armazena dados, é a maetrina das referências.

Atributos:

inicio : Referência para o primeiro nó (cabeça da fila)

fim : Referência para o último nó (cauda da fila)

// Quando a fila é criada, ela está vazia

Construtor():

inicio = NULO

fim = NULO

Mão na Massa: Trabalho Colaborativo

Formem grupos de 2 a 4 estudantes.

Modelar

Desenhar a estrutura
Node e Fila no papel.

Implementar

Codificar adicionar() e
atender().

Testar

Inserir, atender todos,
checar fila vazia.

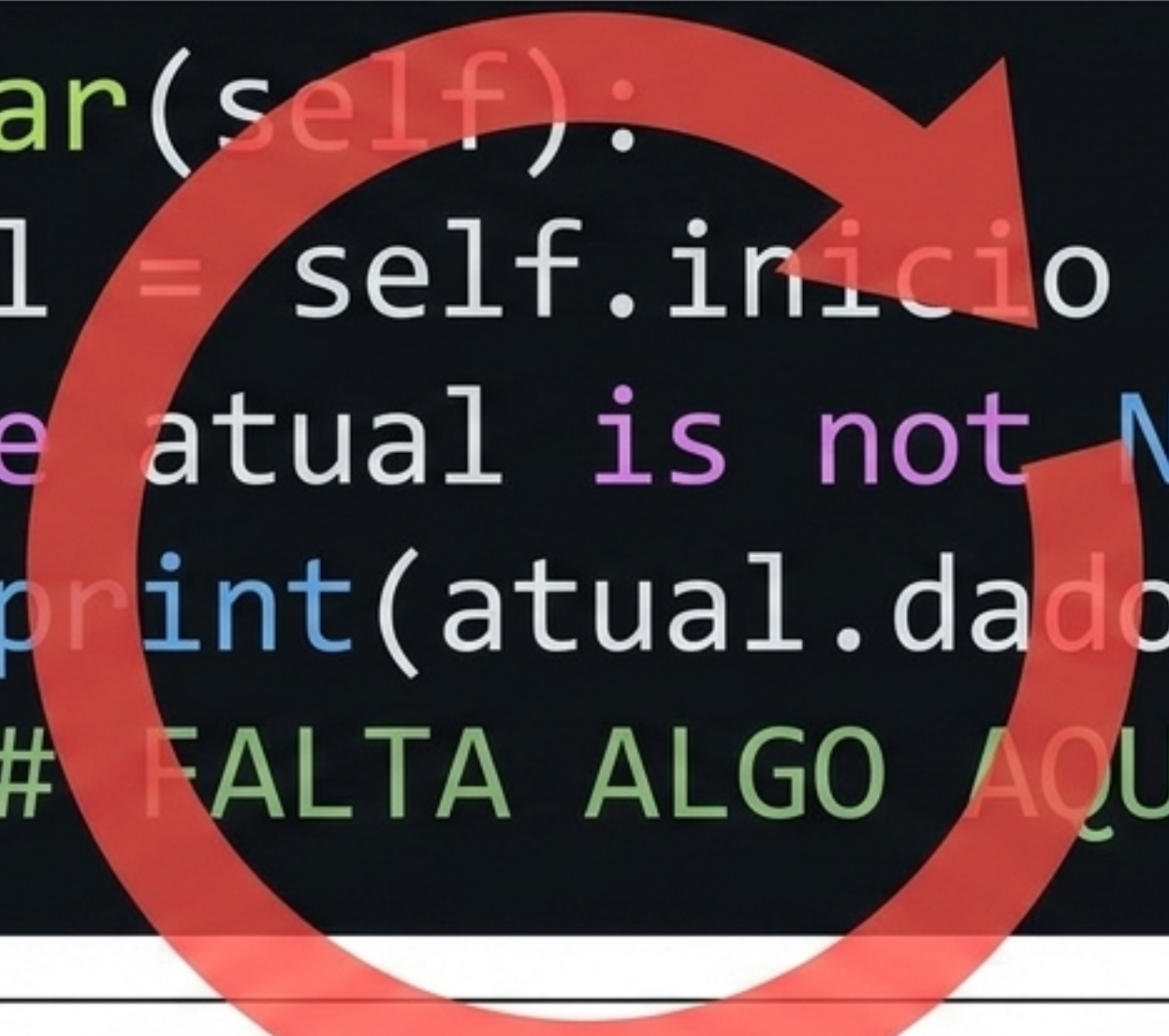
DESAFIO

Como modificar a estrutura para inserir um paciente prioritário antes dos normais, mas mantendo a ordem?

Dica: Pensem em como alterar os fios (referências), não as caixas.

Depuração Analítica: O Laço Infinito

```
def listar(self):  
    atual = self.inicio  
    while atual is not None:  
        print(atual.dado)  
        # FALTA ALGO AQUI!
```



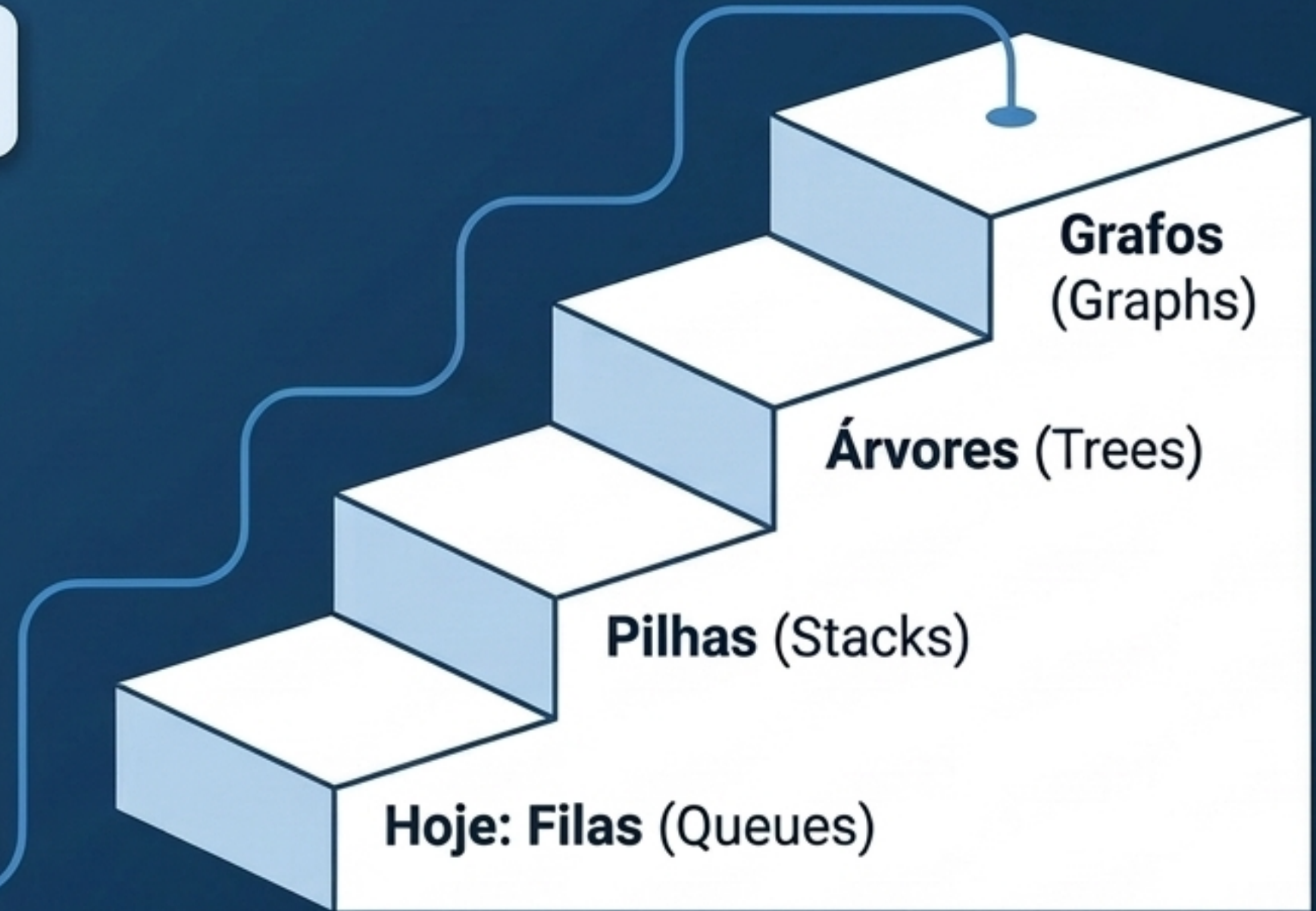
Pensamento Computacional: O que muda dentro do laço? Nada. O laço roda para sempre no mesmo lugar.

A Correção Vital: A etiqueta atual precisa caminhar para frente. Faltou a linha:
atual = atual.proximo.

Próxima aula

O que levamos hoje

- ✓ **id():** O endereço das variáveis revelado.
- ✓ **Referências:** O motor que evita cópias desnecessárias.
- ✓ **Nós e Filas:** Estruturas dinâmicas construídas do zero.



**Pensar em estruturas de dados é pensar em como os fios se conectam.
A base de todo sistema complexo começa em um simples Node.**